

AI Agents

Ronald A. Sumida

rsumida@innovationinsoftware.com



AI Agents

- New way of architecting AI solutions
 - "Microservices for AI"
- Build teams of autonomous agents, each designed to accomplish a single task extraordinarily well
- Use individual agents to solve problems or let them work as a team to solve more complex problems
 - You provide the building blocks
 - AI provides the workflow
- Frameworks provide the messaging infrastructure

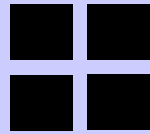


Agentic Frameworks



Agents SDK

Python-first agentic framework from OpenAI. Supports **memory**, **RAG**, **audio/speech**, **tool use**, and more. Includes tools for **Web search**, **file search**, **code generation and execution**, **image generation**, and more.



Agents Framework

Microsoft framework featuring an enterprise-quality graph-based workflow engine that is **LLM-agnostic**. Supports **RAG**, diverse conversation patterns, and more. SDKs available for **Python** and **.NET**.



CrewAI

Open-source Python framework for "crews" of AI agents. **LLM-agnostic** with support for **RAG**, **tool use**, and more. Includes **dozens of built-in tools** and the **richest memory system** of all the frameworks.



Agno

Open-source Python framework that is **LLM-agnostic**. Includes a wide variety of built-in tools and supports **memory**, **persistence**, **RAG**, and more, and offers a playground for **building and testing workflows**.

Creating an Agno Agent

```
from agno.agent import Agent
from agno.models.openai import OpenAIChat

agent = Agent(
    name='Sample Agent',
    instructions='You are a helpful agent who answers questions from users.',
    model=OpenAIChat(id='gpt-5.4-mini')
)

response = agent.run('Why is the sky blue?')
print(response.content)
```

Enforcing a Timeout

```
from agno.agent import Agent
from agno.models.openai import OpenAIChat
import asyncio

agent = Agent(
    name='Sample Agent',
    instructions='You are a helpful agent who answers questions from users.',
    model=OpenAIChat(id='gpt-5.4-mini')
)

try:
    response = await asyncio.wait_for(agent.arun('Why is the sky blue?'), timeout=10)
    print(response.content)
except asyncio.TimeoutError:
    print('Request timed out after 10 seconds')
```

Streaming the Response

```
from agno.agent import Agent
from agno.models.openai import OpenAIChat

agent = Agent(
    name='Sample Agent',
    instructions='You are a helpful agent who answers questions from users.',
    model=OpenAIChat(id='gpt-5.4-mini')
)

response = agent.run('Why is the sky blue?', stream=True)

for chunk in response:
    if chunk.event == 'RunContent':
        print(chunk.content, end='')
```

Using an Azure OpenAI Model

```
from agno.agent import Agent
from agno.models.azure import AzureOpenAI

agent = Agent(
    name='Sample Agent',
    instructions='You are a helpful agent who answers questions from users.',
    model=AzureOpenAI(id='my-gpt-5.4-mini') # Azure deployment name
)

response = agent.run('Why is the sky blue?')
```

Using an Anthropic Model

```
from agno.agent import Agent
from agno.models.anthropic import Claude

agent = Agent(
    name='Sample Agent',
    instructions='You are a helpful agent who answers questions from users.',
    model=Claude(id='claude-haiku-4-5-20251001') # Claude Haiku 4.5
)

response = agent.run('Why is the sky blue?')
```


Using an Ollama Model

```
from agno.agent import Agent
from agno.models.ollama import Ollama

agent = Agent(
    name='Sample Agent',
    instructions='You are a helpful agent who answers questions from users.',
    model=Ollama(id='llama-3.2:3b') # Use the 3B-parameter version of Llama 3.2
)

response = agent.run('Why is the sky blue?')
```

Enabling Chat History

```
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.db.in_memory import InMemoryDb
```

```
agent = Agent(
    name='Sample Agent',
    instructions='You are a helpful agent who answers questions from users',
    model=OpenAIChat(id='gpt-5.4-mini'),
    add_history_to_context=True,
    num_history_runs=10,
    db=InMemoryDb()
)
```

```
response = agent.run('Why is the sky blue?')
```

Persisting Chat History in SQLite

```
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.db.sqlite import SqliteDb

agent = Agent(
    name='Sample Agent',
    instructions='You are a helpful agent who answers questions from users',
    model=OpenAIChat(id='gpt-5.4-mini'),
    add_history_to_context=True,
    num_history_runs=10,
    db=SqliteDb(db_file='data/sessions.db'),
    user_id='stable_user_id' # Required to persist chats on a per-user basis
)

response = agent.run('Why is the sky blue?')
```

Demo

Agents



Tools

- Agno comes with hundreds of tools grouped into "toolkits"

Search

ArxivTools

WebSearchTools

HackerNewsTools

PerplexitySearch

WebSearchTools

WikipediaTools

Data

CsvTools

GoogleBigQueryTools

Neo4jTools

PandasTools

PostgresTools

SQLTools

Social

DiscordTools

GmailTools

RedditTools

SlackTools

TwilioTools

XTools

Others

FileGenerationTools

FirecrawlTools

GoogleCalendarTools

GitHubTools

JiraTools

YFinanceTools

Using the WebSearch Tools

```
from agno.tools.websearch import WebSearchTools

agent = Agent(
    name='Research Agent',
    instructions='You are a helpful research assistant.',
    model=OpenAIChat(id='gpt-5.4-mini'),
    tools=[WebSearchTools()]
)

response = agent.run('Which 10 EU countries had the highest GDP in 2025?')
```

Using the YFinance Tools

```
from agno.tools.yfinance import YFinanceTools

agent = Agent(
    name='Financial Agent',
    instructions='You are a helpful investment analyst',
    model=OpenAIChat(id='gpt-5.4-mini'),
    tools=[YFinanceTools(all=True)] # Include all 9 toolkit functions
)

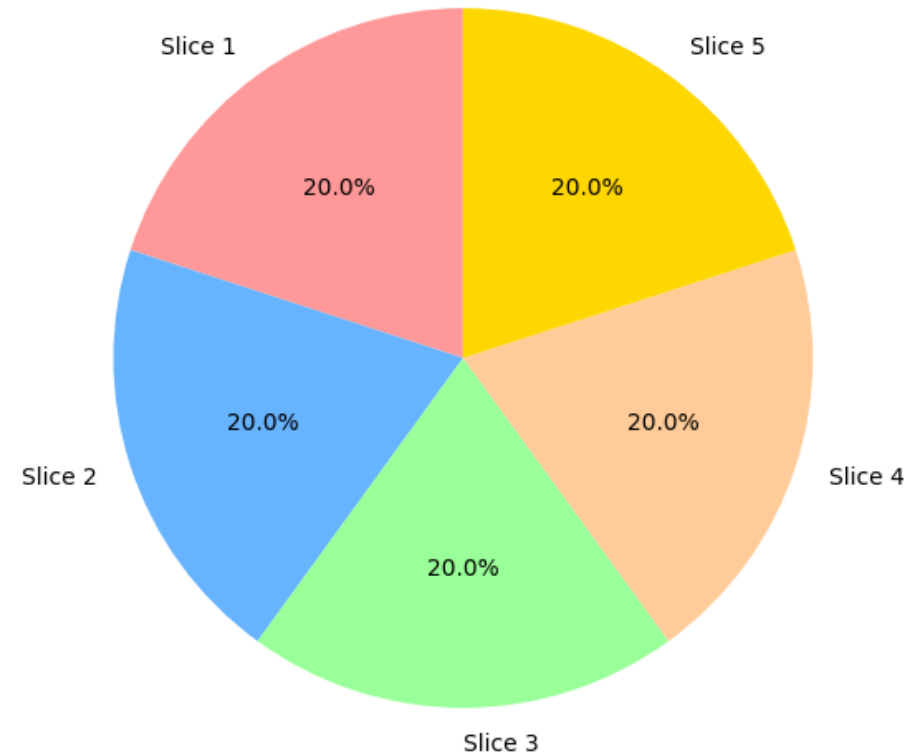
response = agent.run('Summarize analyst recommendations for NVDA')
```

Using the Python Tools

```
from agno.tools.python import PythonTools

agent = Agent(
    name='Visualization Agent',
    instructions='You are a helpful data visualizer',
    model=OpenAIChat(id='gpt-5.4-mini'),
    tools=[PythonTools()]
)

response = agent.run(
    'Generate a pie chart with five equal slices'
)
```

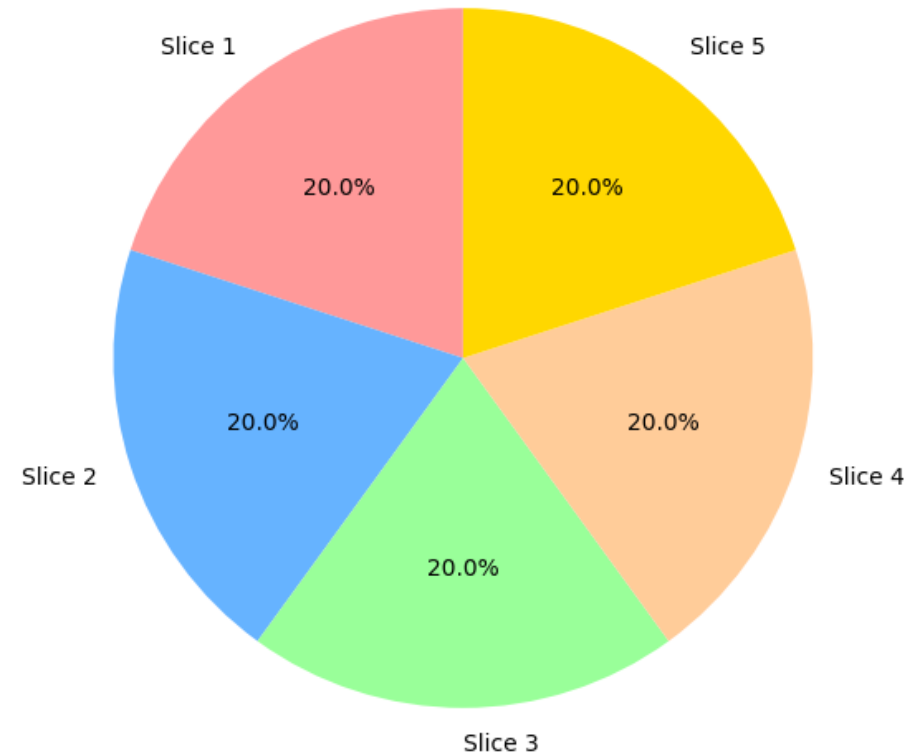


Restricting Python to In-Memory Execution

```
from agno.tools.python import PythonTools

agent = Agent(
    name='Visualization Agent',
    instructions='You are a helpful data visualizer',
    model=OpenAIChat(id='gpt-5.4-mini'),
    tools=[PythonTools(
        include_tools=['run_python_code']
    )]
)

response = agent.run(
    'Generate a pie chart with five equal slices'
)
```



Writing a Custom Tool

```
def get_current_weather(location: str) -> str:
```

```
    '''
```

```
    Retrieves information about the weather at the specified location.
```

```
    Args:
```

```
        location: City or other location whose weather is to be retrieved.
```

```
    Returns:
```

```
        JSON string containing information about the current weather.
```

```
    '''
```

```
    response = requests.get(f'https://api.openweathermap.org/.../weather?q={location}')
```

```
    return json.dumps(response.json())
```

Using a Custom Tool

```
agent = Agent(  
    name='Weather Agent',  
    instructions='You are a helpful agent who provides weather information',  
    model=OpenAIChat(id='gpt-5.4-mini'),  
    tools=[get_current_weather]  
)  
  
response = agent.run('Is it raining in Chicago?')
```

Putting a Human in the Loop

```
@tool(requires_confirmation=True)
```

```
def reply_all(message_id: str, body: str) -> str:
```

```
    '''
```

Sends a Reply All response to an email message.

Args:

message_id: ID of the email message being answered.

body: Text of the reply.

Returns:

Status message indicating that the reply was sent.

```
    '''
```

```
# This code does not run unless the user approves the tool call
```

```
print(f'REPLY ALL to message {message_id}')
```

```
return f'Reply All sent for message {message_id}.'
```

Processing Confirmation Requests

```
response = agent.run('Reply all to this message saying thanks to everyone involved.')
```

```
if response.is_paused:
```

```
    # tools_requiring_confirmation holds one entry per paused tool call
```

```
    for pending_tool in response.tools_requiring_confirmation:
```

```
        print(f'Tool {pending_tool.tool_name} requires confirmation')
```

```
        decision = input('Confirm? (y/n): ').strip().lower()
```

```
        pending_tool.confirmed = decision == 'y'
```

```
    # Resume the paused run. The tool executes now only if approved.
```

```
    response = agent.continue_run(
```

```
        run_id=response.run_id,
```

```
        updated_tools=response.tools
```

```
)
```

Compressing Tool Results

```
from agno.tools.duckduckgo import DuckDuckGoTools

agent = Agent(
    name='Research Agent',
    instructions='You are a helpful research assistant',
    model=OpenAIChat(id='gpt-5.4-mini'),
    tools=[DuckDuckGoTools()],
    compress_tool_results=True
)

response = agent.run('Which 10 EU countries have the highest GDP?')
```

Demo

Agents and Tools



Model Context Protocol (MCP)

- Open standard developed by Anthropic to enable easy integration of language models and external data and APIs
 - Uses client-server architecture with JSON-RPC 2.0 messaging
 - "USB-C port for AI applications"
- Two types of servers
 - **stdio** servers, which run locally in subprocesses of the host
 - **Streamable HTTP** servers, which run remotely and are accessed over HTTP
- Official SDKs available for Python, TypeScript, Java, Rust, C#, and more
- Agno offers built-in support

Connecting Agno to a Stdio MCP Server

```
from agno.tools.mcp import MCPTools

mcp_tools = MCPTools(command=f'npx -y @openbnb/mcp-server-airbnb')
await mcp_tools.connect()

agent = Agent(
    name='Airbnb Agent',
    instructions='You are a helpful agent who has access to Airbnb listings',
    model=OpenAIChat(id='gpt-5.4-mini'),
    tools=[mcp_tools]
)

response = await agent.arun('Where can I stay in Mostar?') # Call arun(), not run()
```

Passing Data in Environment Variables

```
from agno.tools.mcp import MCPTools

mcp_tools = MCPTools(
    command=f'node C:/mcp/flightradar24-mcp-server/dist/index.js',
    env={
        'FR24_API_KEY': os.getenv('FR24_API_KEY'),
        'FR24_API_URL': 'https://fr24api.flightradar24.com'
    }
)

await mcp_tools.connect()
```

Connecting Agno to a Remote MCP Server

```
from agno.tools.mcp import MCPTools

mcp_tools = MCPTools(transport='streamable-http', url='MCP_SERVER_URL')
await mcp_tools.connect()

agent = Agent(
    name='Airbnb Agent',
    instructions='You are a helpful agent who has access to Airbnb listings',
    model=OpenAIChat(id='gpt-5.4-mini'),
    tools=[mcp_tools]
)

response = await agent.arun('Where can I stay in Mostar?') # Call arun(), not run()
```

Passing Data in HTTP Headers

```
from agno.tools.mcp import StreamableHTTPClientParams, MCPTools
```

```
mcp_tools = MCPTools(  
    transport='streamable-http',  
    server_params=StreamableHTTPClientParams(  
        url='MCP_SERVER_URL',  
        headers={  
            'Authorization': 'Bearer YOUR_TOKEN_HERE',  
            'X-Custom-Header': 'value'  
        }  
    )  
)  
  
await mcp_tools.connect()
```

Demo

Agno MCP Clients



Skills

- Provide agents with structured domain expertise
- A **skill** is a self-contained package with:
 - **Instructions** in a SKILL.md file
 - **References** containing additional context supplementing SKILL.md (optional)
 - **Scripts** containing executable code used by the skill (optional)
- Loaded on demand, replacing bulky , monolithic system prompts
 - Agent loads only what it needs, reducing token counts
- Based on the Agent Skills specification at <https://agentskills.io/home>

Code-Security Skill (SKILL.md)

name: security-reviewer

description: Reviews code changes for security vulnerabilities – injection risks, unsafe deserialization, hardcoded secrets, missing input validation, and unsafe eval/exec usage. Use this whenever the user asks for a security review, a vulnerability check, or says "is this code safe," and also proactively whenever reviewing a diff or PR that touches user input, database queries, file paths, subprocess calls, or authentication.

Security Reviewer

Review the given code for exploitable vulnerabilities, not general best practices. Flag only issues an attacker could realistically use.

Checklist

- ****Injection****: string-built SQL, shell commands, or file paths from unsanitized input. Flag ``f"...{user_input}..."`` in queries/commands; prefer parameterized queries or ``subprocess`` argument lists.
- ****Unsafe deserialization****: ``pickle.loads``, ``yaml.load`` (without ``SafeLoader``), or ``eval`/`exec`` on any input that could originate outside the process.
- ****Secrets****: API keys, passwords, or tokens hardcoded in source rather than pulled from environment/secret store.
- ****Missing input validation****: route handlers or public functions that trust request data (type, length, range) without checking it.
- ****Broken auth/access checks****: endpoints that perform privileged actions without verifying the caller is authorized for that specific resource (not just "logged in").

Skills File-System Structure

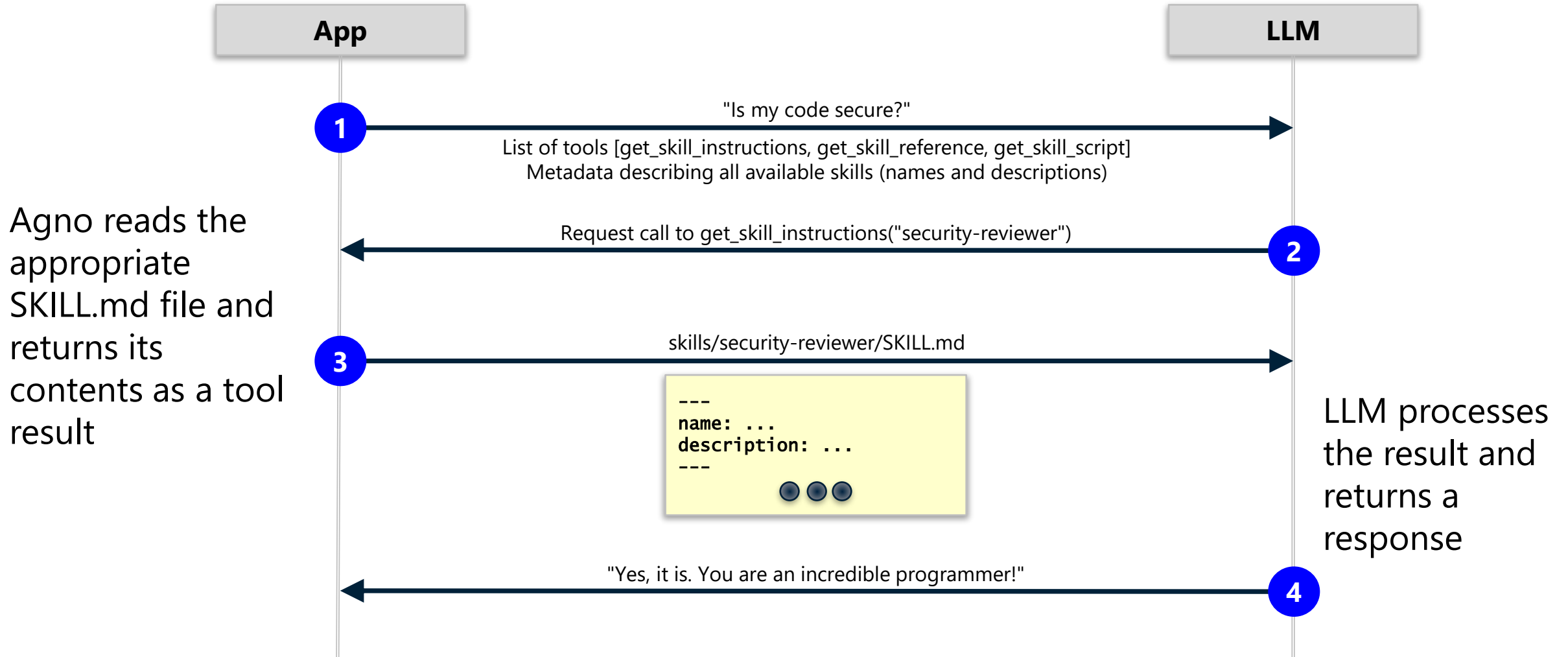
- Root folder serves as container for all skills (e.g., "skills")
- Each skill has its own subfolder with optional **scripts** and **references** subfolders
 - SKILL.md files refer to scripts and reference files when present
- Each skill subfolder name should match "name" in SKILL.md

```
skills/  
├── security-reviewer/  
│   ├── SKILL.md  
├── performance-reviewer/  
│   ├── SKILL.md  
│   └── scripts/  
│       └── run-instrumented.py  
├── style-reviewer/  
│   ├── SKILL.md  
│   └── references/  
│       └── style-guidelines.md  
└── architecture-reviewer/  
    └── SKILL.md
```


Providing Skills to an Agent

```
agent = Agent(  
    name='Code-Review Agent',  
    instructions=''  
        You are a helpful code reviewer who can examine code for security,  
        performance, architecture, and style.  
    ''',  
    model=OpenAIChat(id='gpt-5.4-mini'),  
    skills=Skills(loaders=[LocalSkills('PATH_TO_SKILLS_FOLDER')])  
)
```

How Skills are Loaded



Demo

Agent Skills

